

SKILL WEEK-12:

TASK-1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze 4 Performance

Ans:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#include "executor.h"

void execute_command(Token tokens[], int count)
{
    int commands = 1;

    /* Count pipeline stages */
    for (int i = 0; i < count; i++) {
        if (strcmp(tokens[i].value, "|") == 0) {
            commands++;
        }
    }

    /* Normal command without pipe */
    if (commands == 1) {
        char *args[MAX_TOKENS + 1];

        for (int i = 0; i < count; i++) {
            args[i] = tokens[i].value;
        }

        args[count] = NULL;

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
        }
    }
}
```

```

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());

        execvp(args[0], args);

        perror("Command execution failed");
        exit(EXIT_FAILURE);
    }

    waitpid(pid, NULL, 0);
    printf("Command execution completed.\n");
    return;
}

/* Multiple pipeline commands */
int previous_read = -1;
int token_start = 0;

pid_t pids[MAX_TOKENS];
int pid_count = 0;

for (int i = 0; i <= count; i++) {

    if (i == count || strcmp(tokens[i].value, "|") == 0) {

        int pipefd[2];

```

```
if (i == count || strcmp(tokens[i].value, "|") == 0) {

    int pipefd[2];

    if (i != count) {
        if (pipe(pipefd) == -1) {
            perror("pipe");
            return;
        }
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {

        /* Input from previous pipe */
        if (previous_read != -1) {
            dup2(previous_read, STDIN_FILENO);
            close(previous_read);
        }

        /* Output to next pipe */
        if (i != count) {
            dup2(pipefd[1], STDOUT_FILENO);
            close(pipefd[0]);
            close(pipefd[1]);
        }
    }
}
```

```

        char *args[MAX_TOKENS + 1];

        int j = 0;

        for (int k = token_start; k < i; k++) {
            args[j++] = tokens[k].value;
        }

        args[j] = NULL;

        execvp(args[0], args);

        perror("Command execution failed");
        exit(EXIT_FAILURE);
    }

    pids[pid_count++] = pid;

    if (previous_read != -1) {
        close(previous_read);
    }

    if (i != count) {
        close(pipefd[1]);
        previous_read = pipefd[0];
    }

    token_start = i + 1;
}

if (previous_read != -1) {

```

```

    if (previous_read != -1) {
        close(previous_read);
    }

    /* Wait for every process */
    for (int i = 0; i < pid_count; i++) {
        waitpid(pids[i], NULL, 0);
    }

    printf("Pipeline execution completed.\n");
}

```

```
arikel@DESKTOP-AMUSJME: x + v - □
arikel@DESKTOP-AMUSJME:~/OSSP$ nano src/executor.c
arikel@DESKTOP-AMUSJME:~/OSSP$ gcc -Wall -Wextra -g -Iinclude
rc/*.c -o mysh
arikel@DESKTOP-AMUSJME:~/OSSP$ ./mysh
myShell> ls | grep .c | wc -l

--- Parsed Arguments ---
Argument 1: ls
Argument 2: |
Argument 3: grep
Argument 4: .c
Argument 5: |
Argument 6: wc
Argument 7: -l
6
Pipeline execution completed.
myShell> ls | grep .c | sort | wc -l

--- Parsed Arguments ---
Argument 1: ls
Argument 2: |
Argument 3: grep
Argument 4: .c
Argument 5: |
Argument 6: sort
Argument 7: |
Argument 8: wc
Argument 9: -l
6
Pipeline execution completed.
```

```
myShell> echo Hello World | tr 'a-z' 'A-Z' | wc -c
```

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello
```

```
Argument 3: World
```

```
Argument 4: |
```

```
Argument 5: tr
```

```
Argument 6: 'a-z'
```

```
Argument 7: 'A-Z'
```

```
Argument 8: |
```

```
Argument 9: wc
```

```
Argument 10: -c
```

```
12
```

```
Pipeline execution completed.
```

```
myShell>
```

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Job1

Child PID: 2369

Job1

Command execution completed.

myShell> sleep 1

--- Parsed Arguments ---

Argument 1: sleep

Argument 2: 1

Child PID: 2370

Command execution completed.

myShell> echo Job2

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Job2

Child PID: 2371

Job2

Command execution completed.

myShell> sleep 2

--- Parsed Arguments ---

Argument 1: sleep

Argument 2: 2

Child PID: 2372

Command execution completed.

myShell> ls | nonexistent_command | wc -l

--- Parsed Arguments ---

Argument 1: ls

Argument 2: |

Argument 3: nonexistent_command

Argument 4: |

Argument 5: wc

Argument 6: -l

```
arikela@DESKTOP-AMUSJME: x + v
arikela@DESKTOP-AMUSJME:~/OSSP$ /usr/bin/time -v ./mysh
myShell> ls | grep .c | sort | wc -l

--- Parsed Arguments ---
Argument 1: ls
Argument 2: |
Argument 3: grep
Argument 4: .c
Argument 5: |
Argument 6: sort
Argument 7: |
Argument 8: wc
Argument 9: -l
6
Pipeline execution completed.
myShell> exit
Exiting shell.
Command being timed: "./mysh"
User time (seconds): 0.00
System time (seconds): 0.02
Percent of CPU this job got: 0%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:13.90
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 8168
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 1749
Voluntary context switches: 12
Involuntary context switches: 0
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
```

TASK-2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design 4 Decisions, Review Documentation

Ans:



arikela@DESKTOP-AMUSJME: ×



GNU nano 8.7.1

docs/design.md *

OSSP Shell Project - Architecture and Design

1. Project Architecture

The shell is implemented using separate modules for parsing, command execution, built-in commands, and process management.

Project flow:

User Input

|

v

main.c

|

v

parser.c

|

v

Token Array

|

v

executor.c

|

|

v

Normal Command

|

fork()

|

execvp()

|

waitpid()

|

v

Pipeline

|

pipe()

|

fork()

|

dup2()

|

execvp()

waitpid()

execvp()

waitpid()

2. Module Description

main.c

Controls the main shell loop. It reads commands from the user, passes them to the parser, and sends the parsed tokens to the executor.

parser.c

Tokenizes the user command into individual arguments.

For example:

```
ls | grep .c | wc -l
```

is converted into:

```
ls
|
grep
.c
|
wc
-l
```

executor.c

Executes external commands using fork() and execvp(). It also implements pipelines using pipe(), dup2(), fork(), execvp(), and waitpid().

builtin.c

Handles shell built-in commands that must be executed by the shell process itself.

3. Execution Path

For a normal command:

User Command

|

v

Parser

|

v

Tokens

|

v

fork()

|

v

Child Process

|

v

execvp()

|

v

Command Execution

|

v

Parent uses waitpid()

For a pipeline:

Command 1 | Command 2 | Command 3

Command 1

|

v

Pipe 1

|

v

Command 2

|

v

Pipe 2

|

v

Command 3

Each pipeline stage is executed as a separate child process.

4. System Calls

fork()

Creates a new child process.

execvp()

Replaces the child process with the requested program.

waitpid()

Allows the parent shell to wait for child processes to finish.

pipe()

Creates a communication channel between processes in a pipeline.

Creates a communication channel between processes in a pipeline.

dup2()

Redirects standard input or output to a pipe.

5. Design Decisions

1. The parser and executor are separated into different modules to improve maintainability.
2. Header files are stored separately in the include directory.
3. External commands are executed using child processes so that the shell process remains active.
4. waitpid() is used to synchronize the parent shell with its child processes.
5. Pipes and dup2() are used to implement communication between pipeline stages.
6. Error messages are displayed when command execution fails.
7. Multiple pipeline stages are supported by creating multiple child processes and connecting them through pipes.

6. Error Handling

The shell handles errors from system calls and command execution. If a command does not exist, an error message is displayed and the shell continues running.

Example:|

The shell handles errors from system calls and command execution. If a command does not exist, an error message is displayed and the shell continues running.

Example:

Command execution failed: No such file or directory

The shell can then accept another command.

7. Testing

The shell was tested using:

```
echo Hello
pwd
ls
invalid_command
ls | grep .c
ls | grep .c | wc -l
ls | grep .c | sort | wc -l
```

The multi-stage pipeline tests completed successfully.

8. Conclusion

The shell uses a modular architecture combining command parsing, process creation, command execution, synchronization, and inter-process communication. The design allows normal commands and multi-stage pipelines to be executed while maintaining error handling and process control.

```

arikel@DESKTOP-AMUSJME:~/OSSP$ cat README.md
arikel@DESKTOP-AMUSJME:~/OSSP$ nano docs/design.md
arikel@DESKTOP-AMUSJME:~/OSSP$ cat docs/design.md
# OSSP Shell Project - Architecture and Design

## 1. Project Architecture

The shell is implemented using separate modules for parsing,
command execution, built-in commands, and process management.

Project flow:

User Input
  |
  v
main.c
  |
  v
parser.c
  |
  v
Token Array
  |
  v
executor.c
  |
  +-----+
  |       |
  |       |
  v       v
Normal Command Pipeline
  |       |
  fork()  pipe()

```

TASK-3: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
arikel@DESKTOP-AMUSJME:~/OSSP$ find . -maxdepth 2 -type f | sort
./
./git/COMMIT_EDITMSG
./git/HEAD
./git/config
./git/description
./git/index
./gitignore
./Makefile
./README.md
./assets/architecture.png
./bin/my_shell
./docs/design.md
./docs/report.pdf
./include/builtin.h
./include/executor.h
./include/parser.h
./include/shell.h
./mysh
./scripts/run.sh
./src/builtin.c
./src/executor.c
./src/main.c
./src/parser.c
./src/shell.c
./task1
./task1.c
./task2
./task2.c
./test_results.txt
./test_shell.sh
./tests/test_executor.c
./tests/test_parser.c
arikel@DESKTOP-AMUSJME:~/OSSP$ ls -la
total 128
drwxr-xr-x 12 arikela arikela 4096 Oct  8 05:31 .
```

```
arikela@DESKTOP-AMUSJME: x + v
arikela@DESKTOP-AMUSJME:~/OSSP$ ls -la
total 128
drwxr-xr-x 12 arikela arikela 4096 Oct  8 05:31 .
drwxr-x--- 18 arikela arikela 4096 Sep 17 09:46 ..
drwxr-xr-x  7 arikela arikela 4096 Aug 11 09:30 .git
-rw-r--r--  1 arikela arikela    0 Aug  1 07:01 .gitignore
-rw-r--r--  1 arikela arikela    0 Aug  1 06:57 Makefile
drwxr-xr-x  2 arikela arikela 4096 Aug  1 03:11 OSSP
-rw-r--r--  1 arikela arikela    0 Aug  1 06:57 README.md
drwxr-xr-x  2 arikela arikela 4096 Jul 30 09:21 assets
drwxr-xr-x  2 arikela arikela 4096 Jul 30 09:14 bin
drwxr-xr-x  2 arikela arikela 4096 Oct  8 05:49 docs
drwxr-xr-x  2 arikela arikela 4096 Aug 11 07:06 include
-rwxr-xr-x  1 arikela arikela 23224 Oct  8 05:31 mysh
drwxr-xr-x  2 arikela arikela 4096 Jul 30 09:10 obj
drwxr-xr-x  2 arikela arikela 4096 Jul 30 09:19 scripts
drwxr-xr-x  2 arikela arikela 4096 Oct  8 05:30 src
-rwxr-xr-x  1 arikela arikela 16536 Aug 22 08:11 task1
-rw-r--r--  1 arikela arikela  3211 Aug 22 08:11 task1.c
-rwxr-xr-x  1 arikela arikela 18768 Aug 22 08:18 task2
-rw-r--r--  1 arikela arikela  2397 Aug 22 08:18 task2.c
-rw-r--r--  1 arikela arikela   917 Oct  8 03:38 test_results.tx
t
-rwxr-xr-x  1 arikela arikela   332 Oct  8 03:38 test_shell.sh
drwxr-xr-x  2 arikela arikela 4096 Jul 30 09:18 tests
arikela@DESKTOP-AMUSJME:~/OSSP$ gcc -Wall -Wextra -g -Iinclude s
rc/*.c -o mysh
arikela@DESKTOP-AMUSJME:~/OSSP$ ./mysh
myShell> echo Task3

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Task3
Child PID: 2584
Task3
Command execution completed.
myShell> pwd
```

```

arikel@DESKTOP-AMUSJME:~/OSSP$ ./mysh
myShell> echo Task3

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Task3
Child PID: 2584
Task3
Command execution completed.
myShell> pwd

--- Parsed Arguments ---
Argument 1: pwd
Child PID: 2585
/home/arikel/OSSP
Command execution completed.
myShell> ls

--- Parsed Arguments ---
Argument 1: ls
Child PID: 2586
Makefile  assets  include  scripts  task1.c  test_results.txt
OSSP      bin     mysh     src       task2    test_shell.sh
README.md docs    obj      task1     task2.c  tests
Command execution completed.
myShell> exit
Exiting shell.
arikel@DESKTOP-AMUSJME:~/OSSP$ ls src include docs
docs:
design.md  report.pdf

include:
builtin.h executor.h parser.h  shell.h

src:
builtin.c executor.c main.c  parser.c  shell.c

```

```

arikel@DESKTOP-AMUSJME:~/OSSP$ ls -lh mysh docs/design.md docs/
report.pdf
-rw-r--r-- 1 arikela arikela 3.4K Oct  8 05:49 docs/design.md
-rw-r--r-- 1 arikela arikela   0 Jul 30 09:15 docs/report.pdf
-rwxr-xr-x 1 arikela arikela 23K Oct  8 05:53 mysh
arikel@DESKTOP-AMUSJME:~/OSSP$

```

TASK-4: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
arikel@DESKTOP-AMUSJME:~/OSSP$ ./mysh
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child PID: 2635
Hello
Command execution completed.
myShell> pwd

--- Parsed Arguments ---
Argument 1: pwd
Child PID: 2636
/home/arikel/OSSP
Command execution completed.
myShell> ls

--- Parsed Arguments ---
Argument 1: ls
Child PID: 2637
Makefile  assets  include  scripts  task1.c  test_results.txt
OSSP      bin     mysh     src       task2    test_shell.sh
README.md docs    obj      task1     task2.c  tests
Command execution completed.
myShell> exit
Exiting shell.
arikel@DESKTOP-AMUSJME:~/OSSP$ ./mysh
myShell> ls | grep .c

--- Parsed Arguments ---
Argument 1: ls
Argument 2: |
Argument 3: grep
Argument 4: .c
docs
```

```
arikel@DESKTOP-AMUSJME:~/OSSP$ ./mysh
```

```
myShell> ls | grep .c
```

```
--- Parsed Arguments ---
```

```
Argument 1: ls
```

```
Argument 2: |
```

```
Argument 3: grep
```

```
Argument 4: .c
```

```
docs
```

```
include
```

```
scripts
```

```
src
```

```
task1.c
```

```
task2.c
```

```
Pipeline execution completed.
```

```
myShell> ls | grep .c | sort | wc -l
```

```
--- Parsed Arguments ---
```

```
Argument 1: ls
```

```
Argument 2: |
```

```
Argument 3: grep
```

```
Argument 4: .c
```

```
Argument 5: |
```

```
Argument 6: sort
```

```
Argument 7: |
```

```
Argument 8: wc
```

```
Argument 9: -l
```

```
6
```

```
Pipeline execution completed.
```

```
myShell> echo Hello World | wc -c
```

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello
```

```
Argument 3: World
```

```
Argument 4: |
```

```
arikela@DESKTOP-AMUSJME:~/OSSP$ ./mysh  
myShell> ls | grep .c | sort | wc -l
```

```
--- Parsed Arguments ---
```

```
Argument 1: ls  
Argument 2: |  
Argument 3: grep  
Argument 4: .c  
Argument 5: |  
Argument 6: sort  
Argument 7: |  
Argument 8: wc  
Argument 9: -l  
6
```

```
Pipeline execution completed.
```

```
myShell> ls | grep .h | sort | wc -l
```

```
--- Parsed Arguments ---
```

```
Argument 1: ls  
Argument 2: |  
Argument 3: grep  
Argument 4: .h  
Argument 5: |  
Argument 6: sort  
Argument 7: |  
Argument 8: wc  
Argument 9: -l  
2
```

```
Pipeline execution completed.
```

```
myShell> exit
```

```
Exiting shell.
```

```
arikela@DESKTOP-AMUSJME:~/OSSP$
```